Universidade Tuiuti do Paraná

Curso: Ciência da Computação

Estudo Dirigido de Programação Avançada

Gustavo Coraleski

Relatório Técnico: Desenvolvimento do Sistema "Battle Chip"

1 - Resumo /Objetivo

O objetivo deste projeto foi desenvolver um simulador do jogo Batalha Naval utilizando a linguagem Java. O foco principal foi aplicar conceitos avançados de Programação Orientada a Objetos (POO) como vistos nas aulas, como herança, polimorfismo, encapsulamento e manipulação de referências de memória para controle de estado do jogo.

2- Introdução

O presente projeto consiste no desenvolvimento do sistema **"Battle Chip"**, um simulador do clássico jogo Batalha Naval implementado na linguagem **Java**. A aplicação permite partidas em dois modos distintos: entre dois jogadores humanos (1 vs 1) ou contra a inteligência artificial (1 vs PC).

O principal desafio técnico e objetivo deste estudo foi o gerenciamento de **tabuleiros simultâneos por jogador**: um tabuleiro real, contendo a posição estratégica e oculta dos navios, e um tabuleiro de visualização (template), utilizado para o feedback das jogadas (acertos e erros). Este mecanismo exigiu uma lógica de dano persistente e sincronizada, garantindo que as interações entre as diferentes classes do sistema mantivessem a integridade do estado da partida.

Para a conclusão deste projeto, foram aplicados conceitos fundamentais de **Programação Orientada a Objetos (POO)**, tais como herança para a criação da frota de navios, encapsulamento para a segurança dos dados dos jogadores e a manipulação de referências de memória para garantir que os objetos atingidos no tabuleiro refletissem corretamente o status de vida nas instâncias originais.

3 – Descrição do código / Metodologia

Estruturas Criadas:

- **Classe Navio:** Define os atributos fundamentais (vida, tamanho, tipo). Utiliza herança para classes específicas como Submarino, PortaAviao, etc.
- **Classe Tabuleiro:** Gerencia uma matriz de caracteres para exibição e uma matriz de objetos do tipo (Navio) para controle de colisões e dano.
- **Classe Jogador:** Agrega um tabuleiro e uma "frota" (container de navios), permitindo o gerenciamento individual de vidas.
- **Classe Jogo:** Concentra a máquina de estados, controlando o fluxo de turnos, validação de entradas e condições de vitória.

Funções Principais:

- **posicionaBarco()**: Realiza a inserção dos navios validando limites da matriz e sobreposições.
- **atacarTabuleiro()**: Função central que recebe coordenadas, verifica acerto/erro, atualiza a matriz de visualização e, via referência, decrementa a vida do objeto navio atingido.
- **preparaJogador()**: Sobrecarga de método que permite tanto a entrada manual via Scanner quanto a geração aleatória de posições para a IA.

4– Discussão/ Análise

- **Decisões de Implementação**:

Para a lógica dos tabuleiros e como gerenciar os ataques, eu decidi separar as coisas logo no início. Quando o jogo começa, ele já cria o jogador (com seu tabuleiro e seus navios reais) e também uma instância de "matriz template" para cada um, que serve como um modelo para esconder as informações do oponente:

```
jogador1 = new Jogador(new Tabuleiro(), new Navio());
jogador2 = new Jogador(new Tabuleiro(), new Navio());
jogador1MatrizTemplate = new Jogador(new Tabuleiro());
jogador2MatrizTemplate = new Jogador(new Tabuleiro());
```

Na hora de posicionar os barcos com o método posicionaBarco(), o sistema busca o navio na frota que foi criada para aquele jogador. A sacada aqui foi usar duas matrizes trabalhando juntas: a **matriz principal**, que guarda os caracteres 'B' para mostrar visualmente o tamanho do navio, e a matriz **gradeNavios**, que armazena o endereço de memória (a referência) do objeto navio em cada posição ocupada por ele.

Eu cheguei nessa solução porque, no começo, eu estava criando instâncias novas de navio toda vez que buscava o nome. Isso era um problema: se eu acertasse uma ponta do porta-aviões, ele perdia 1 de vida (de 5 para 4), mas se eu acertasse

outra parte dele, a vida voltava para 5 e caía para 4 de novo, porque era um objeto novo em cada casa. Guardando o **endereço de memória** na gradeNavios, todas as casas do navio apontam para o mesmo objeto. Assim, se eu tirar vida de uma parte, o navio inteiro sente o dano.

Além disso, a matriz gradeNavios é fundamental para a lógica de vitória. Eu cheguei a pensar em algo mais simples, como contar se apareceram 15 caracteres '@' no tabuleiro do oponente (que é a soma de todas as vidas), mas como eu criei uma classe Navio com atributos próprios, eu queria usar o potencial da Orientação a Objetos. Por isso, preferi usar a matriz de instâncias para descontar a vida real de cada navio.

Já na lógica de ataque, o funcionamento é o seguinte: quando um jogador atira, eu checo as coordenadas na matriz principal e na matriz de modelo do defensor. Se na principal tiver um 'B', eu marco um '@' (navio atingido) tanto no modelo do atacante quanto no tabuleiro real do defensor. Ao mesmo tempo, busco o objeto exato na gradeNavios para diminuir a vida dele de verdade.

No fim das contas, a tela mostra duas coisas: o tabuleiro real do jogador da vez (com seus barcos aparecendo) e o tabuleiro de modelo do oponente (onde os barcos ficam escondidos e só aparecem os tiros na água '~' ou os acertos '@'), tudo sincronizado pela gradeNavios para controlar a vida da frota.

Na lógica de ataque da IA eu só fiz com que ela não atire na mesma posição que já tinha atirado, para ela ficar um pouco mais inteligente.

- **Limitações:**
  - A interface é baseada em console (CLI), o que limita a experiência visual, mas foca na robustez da lógica.
  - A IA utiliza um algoritmo de busca aleatória simples com validação de tiro na mesma posição, sem lógica de perseguição após o primeiro acerto.

- **Possíveis melhorias:**
  - Implementação de uma Interface Gráfica (Swing ou JavaFX).
  - Melhoria na IA para que, ao acertar um barco, ela realize buscas em cruz (adjacências) para afundar o navio mais rapidamente.

5 – Conclusão

O desenvolvimento do Battle Chip demonstrou a importância da organização estrutural em Java. A separação clara entre as entidades (Navio, Tabuleiro e Jogador) facilitou a depuração e a implementação de regras de negócio complexas, como o controle de turnos e a verificação automática de vitória ao zerar as vidas da frota adversária.

6 - Referências

- Aulas de Programação avançada – Luiz Adolpho Baroni

7- Apêndice

Contribuição dos Integrantes:
- Gustavo Coraleski